IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Tsantalis Ioannis
February 24, 2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
  - Data Collection through API
  - Data Collection with Web Scraping
  - Data Wrangling
  - Exploratory Data Analysis with SQL
  - Exploratory Data Analysis with Data Visualization
  - Interactive Visual Analytics with Folium
  - Machine Learning Prediction
- Summary of all results
  - Exploratory Data Analysis result
  - Interactive Analytics in screenshots
  - Predictive Analytics result from Machine Learning Lab

Introduction

- Project background and context

  SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other provides cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore, if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch. This goal of the project is to create a machine learning pipeline to predict if the first stage will land successfully.

- Problems you want to find answers

  - What factors determine if the rocket will land successfully?
  - The interaction amongst various features that determine the success rate of a successful landing
  - What operating conditions needs to be in place to ensure a successful landing program

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Data was collected using SpaceX REST API and web scraping from Wikipedia
- Perform data wrangling
  - One-hot encoding was applied to categorical features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - How to build, tune, evaluate classification models

Data Collection

Data collection is the process of gathering and measuring information. The dataset was collected by SpaceX REST API and Web Scraping from Wikipedia.

For REST API, it’s started by using the get request. Then we decoded the response as Json and turn it into a pandas dataframe using json_normalize(). We then cleaned the data, checked for missing values and fill whatever needed.

For Web Scraping, we used the BeautifulSoup to extract HTML table, parse the table and convert it to a pandas dataframe for further analysis.

Data Collection – SpaceX API

- Get request for rocket launch data using REST API
- Use json_normalize method to convert json result to dataframe
- Performed data cleaning and filling the missing value
- https://github.com/gtsantalis/datasciencecapstone/blob/main/jupyter-labs-spacex-data-collection-api.ipynb

In [9]:
```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

In [10]:
```
response = requests.get(spacex_url)
```

In [12]:
```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_a
```

We should see that the request was successfull with the 200 status response code

In [13]:
```
response=requests.get(static_json_url)
```

In [14]:
```
response.status_code
```

Out[14]:
```
200
```

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

In [15]:
```
# Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

In [17]:
```
# Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have multiple paylo
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

Data Collection - Scraping

- Request the Falcon9 Launch Wiki page from url
- Create a BeatifulSoup from the HTML response
- Extract all column/variable names from the HTML header
- https://github.com/gtsantalis/datasciencecapstone/blob/main/jupyter-labs-webscraping.ipynb

In [4]:
```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) "
                  "Chrome/91.0.4472.124 Safari/537.36"
}
```

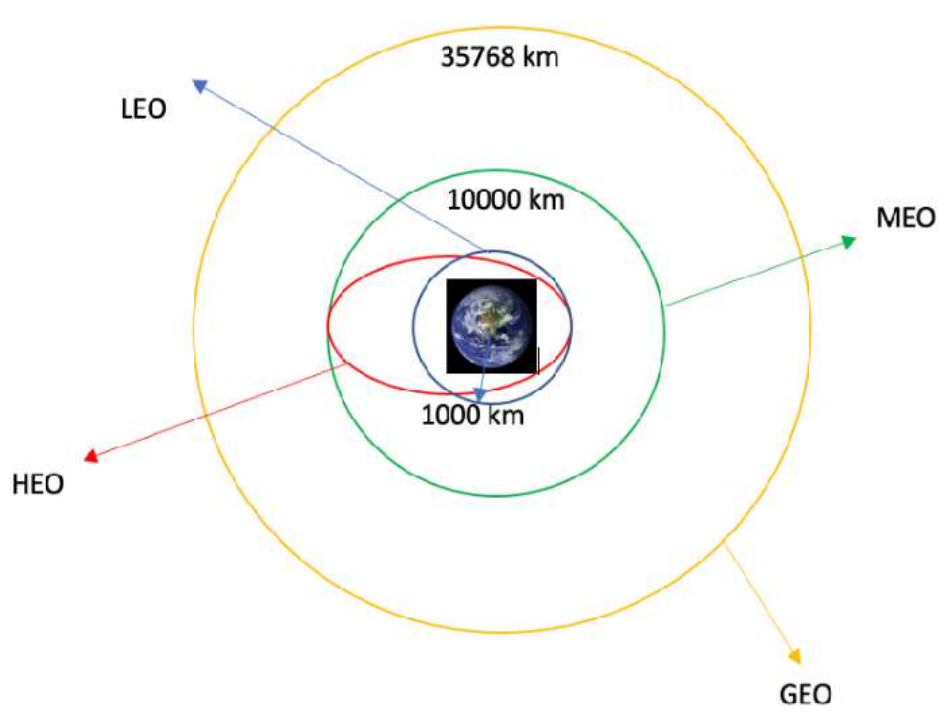
In [5]:
```
# use requests.get() method with the provided static_url and headers
# assign the response to a object
html_data = requests.get(static_url, headers=headers)
```

Create a `BeautifulSoup` object from the HTML `response`

In [6]:
```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(html_data.text, 'html.parser')
```

In [19]:
```
extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable plainrowheaders collapsible")):
   # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
        else:
            flag=False
        #get table element
        row=rows.find_all('td')
        #if it is number save cells in a dictonary
```

Data Wrangling



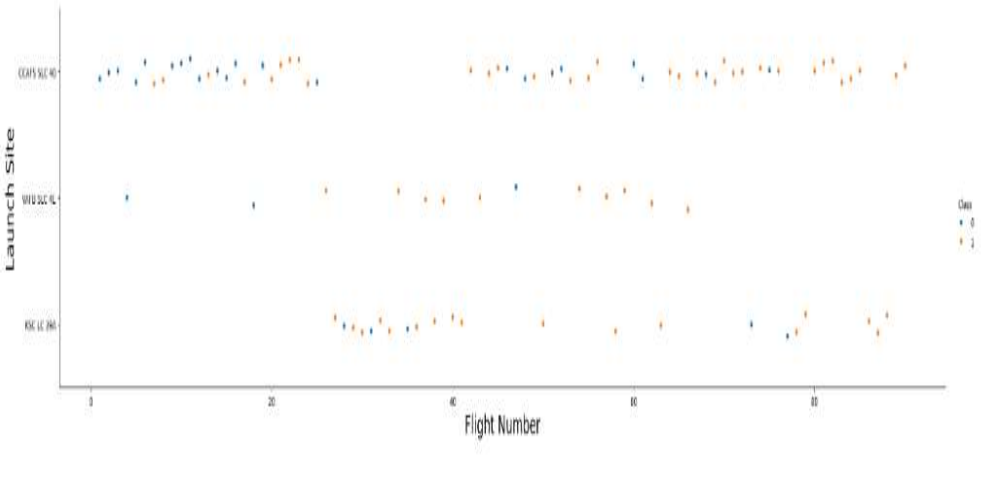
https://github.com/gtsantalis/datasciencecapstone/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb

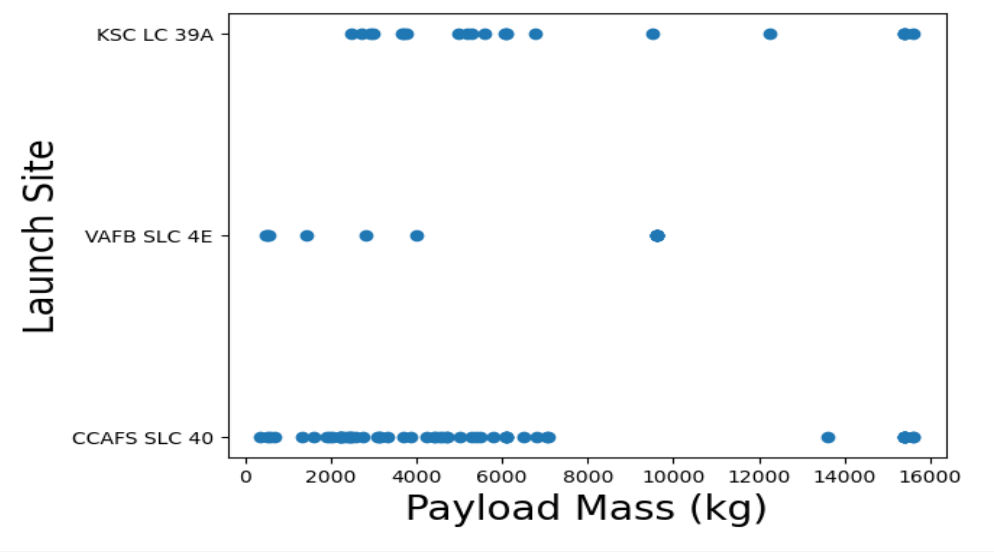
Data Wrangling is the process of cleaning and unifying messy and complex data for easy access and Exploratory Data Analysis.

We calculated the number of launches on each site, then calculated the number and occurrence of mission outcome per orbit type.

We then created a landing outcome label from the outcome column. Last, we exported the result to a CSV.

EDA with Data Visualization





We started by using scatter graph to find the relationship between the attributes such as between:

- Payload and Flight Number
- Launch Site and Flight Number
- Launch Site and Payload
- Success Rate and Orbit
- Flight Number and Orbit
- Payload and Orbit

https://github.com/gtsantalis/datasciencecapstone/blob/main/edadataviz.ipynb

EDA with SQL

Using SQL, we had performed many queries to get better understanding of the dataset.

- Displaying the names of the launch sites
- Displaying 5 records where launch sites begin with the string CCA
- Displaying the total payload mass carried by booster version F9 v1.1
- Listing the date when the first successful landing outcome in ground pad was achieved
- Listing the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
- Listing the total number of successful and failure mission outcomes
- Listing the names of the booster_versions which have carried the maximum payload mass
- Listing the failed landing_outcomes or success between the date 2010-06-04 and 2017-03-20 in descending order

https://github.com/gtsantalis/datasciencecapstone/blob/main/jupyter-labs-eda-sql-coursera_sqllite.ipynb

Build an Interactive Map with Folium

To visualize the launch data into an interactive map, we took the latitude and longitude coordinates at each launch site and added a circle marker around each launch site with a label of the name from the launch site.

We then assigned the dataframe launch_outcomes for failure(class 0) to Red marker and success(class 1) to Green marker on the map in MarkerCluster.

We then used the Haversine's formula to calculated the distance of the launch sites to various landmark to find answers to the questions of:

- How close the launch sites with railways, highways and coastlines
- How close the launch sites with nearby cities

https://github.com/gtsantalis/datasciencecapstone/blob/main/lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

- We built an interactive dashboard with Plotly dash which allowing the user to play around with the data as they need
- We plotted pie charts showing the total launches by a certain sites
- We then plotted scatter graph showing the relationship with Outcome and Payload Mass for the different booster version

https://github.com/gtsantalis/datasciencecapstone/blob/main/spacex-dash-app.py

Predictive Analysis (Classification)

Building the Model

- Load the dataset into NumPy and Pandas
- Transform the data and then split into training and test datasets
- Decide which type of ML to use
- Set the parameters and algorithms to GridSearchCV and fit it to dataset

Evaluating the Model

- Check the accuracy for each model
- Get tuned hyperparameters for each type of algorithms
- Plot the confusion matrix

Improving the Model

- Use Feature Engineering and Algorithm Tuning

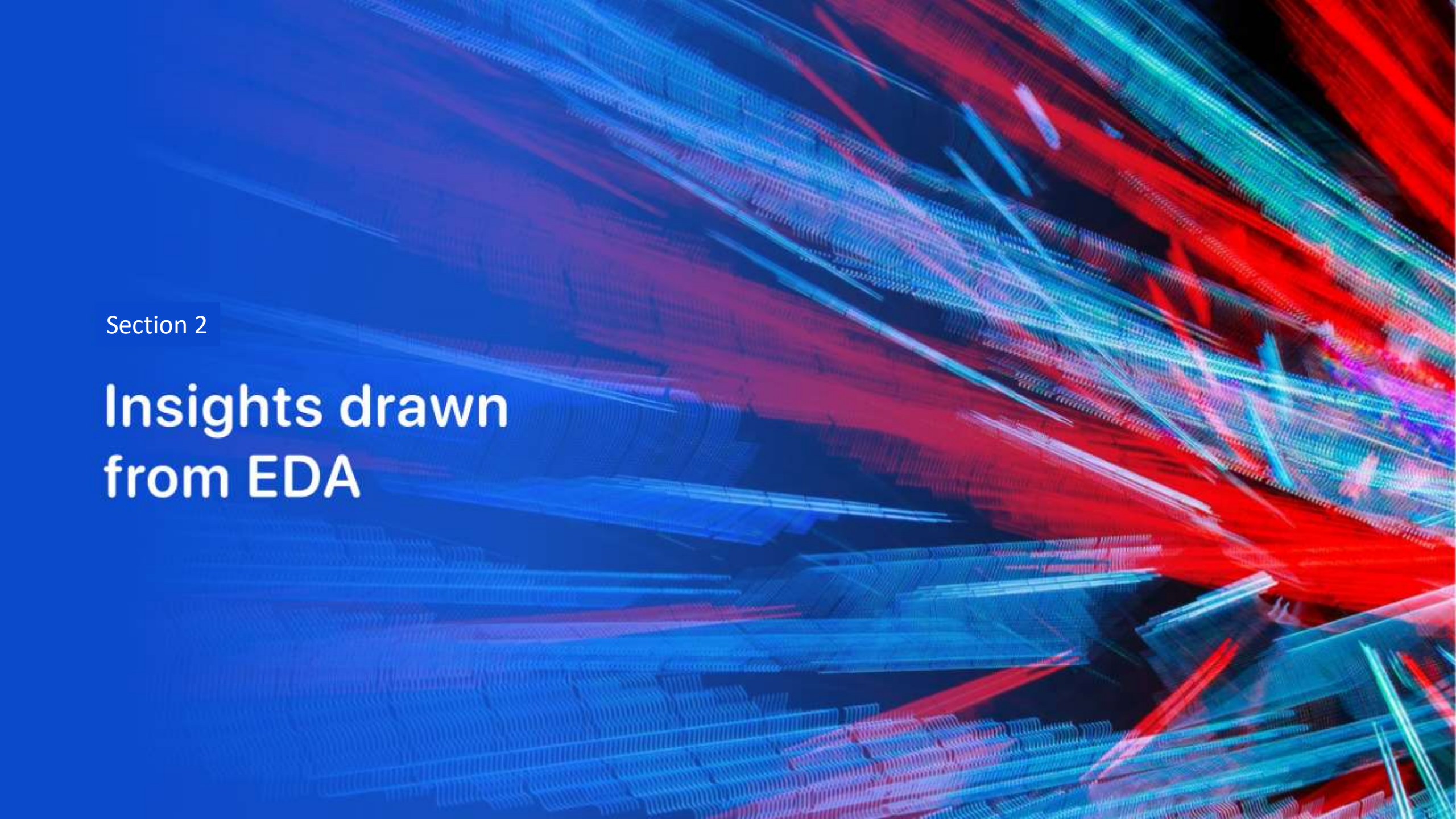
Find the Best Model

- The model with the best accuracy score will be the best performing model

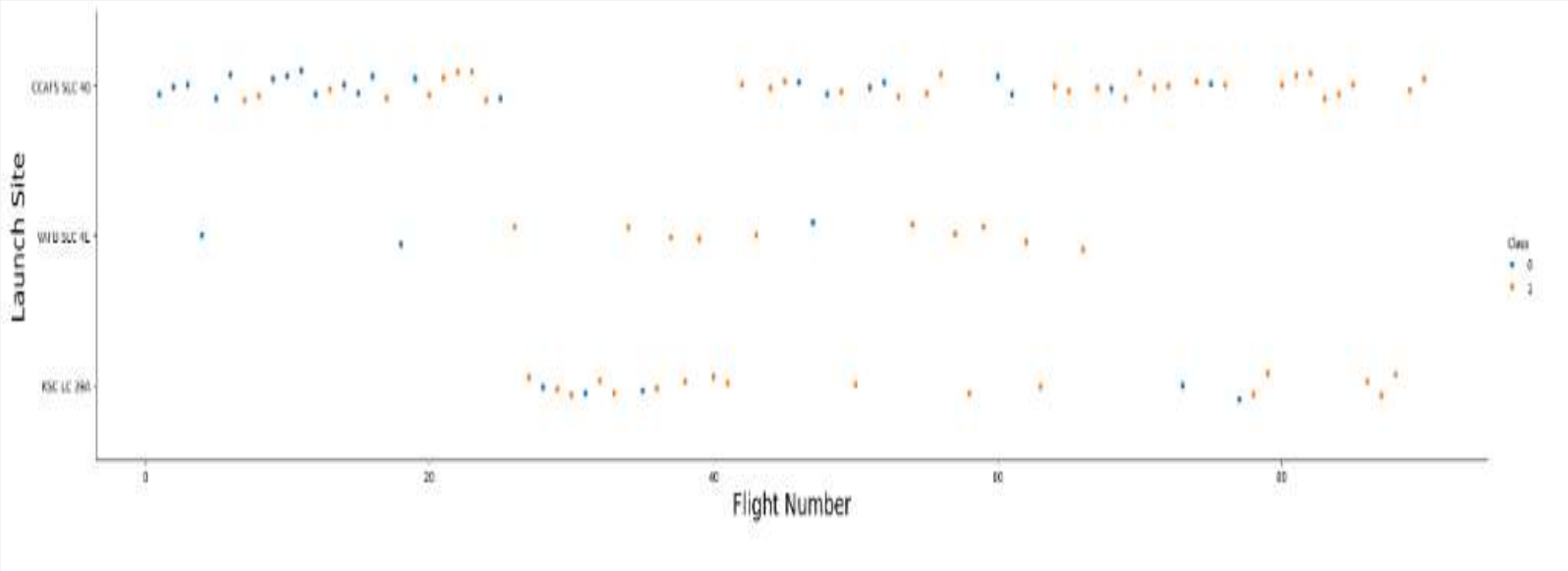
https://github.com/gtsantalis/datasciencecapstone/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results


Section 2
Insights drawn
from EDA

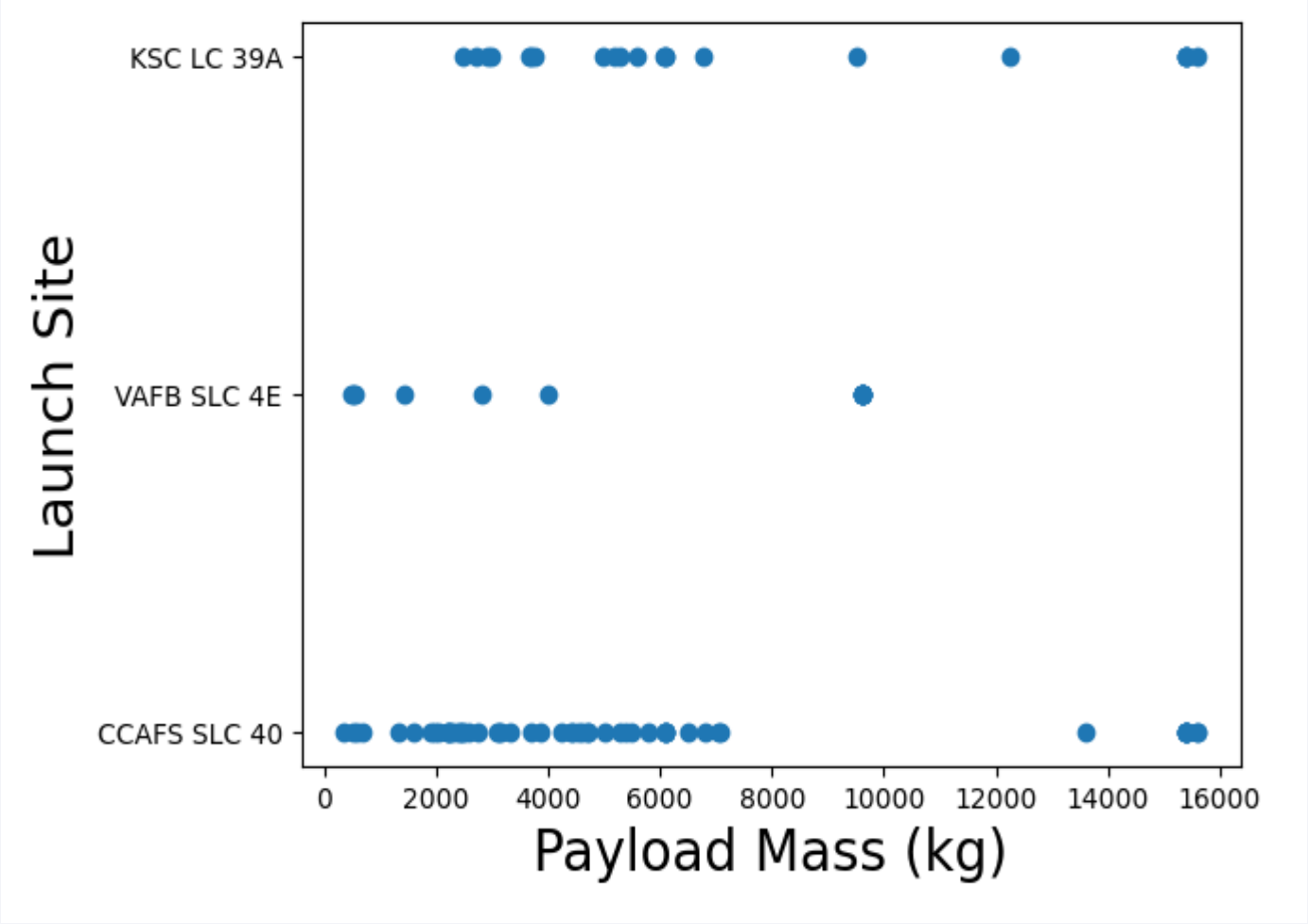
Flight Number vs. Launch Site



This scatter plot shows that the larger the flights amount of the launch site, the greater the success rate will be.

However, site CCAFS SLC40 shows the least pattern of this.

Payload vs. Launch Site



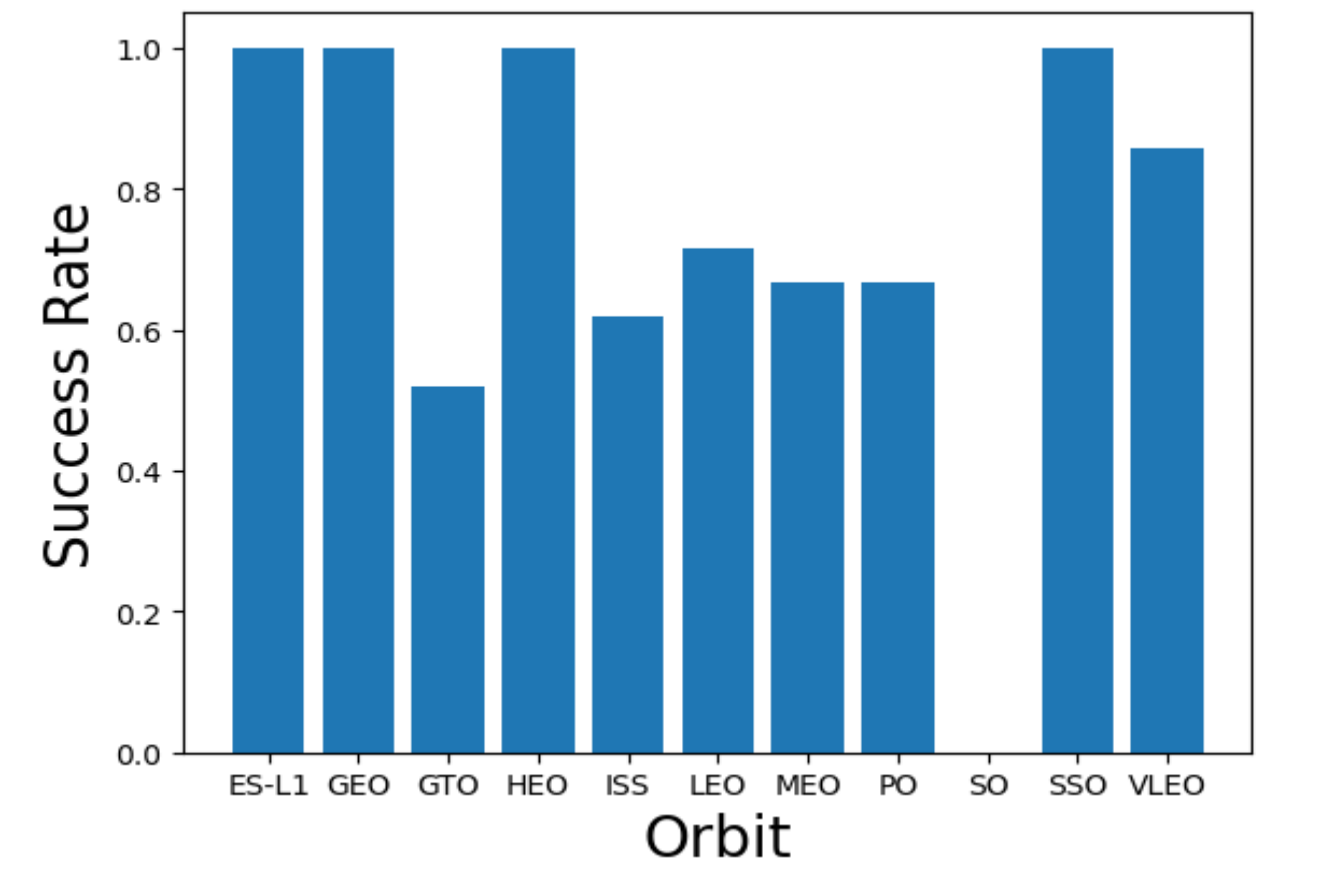
This scatter plot shows once the payload mass is greater than 7000kg, the probability of the success rate will be highly increased.

However, there is no clear pattern to say the launch site is dependent to the paylaod mass for the success rate.

Success Rate vs. Orbit Type

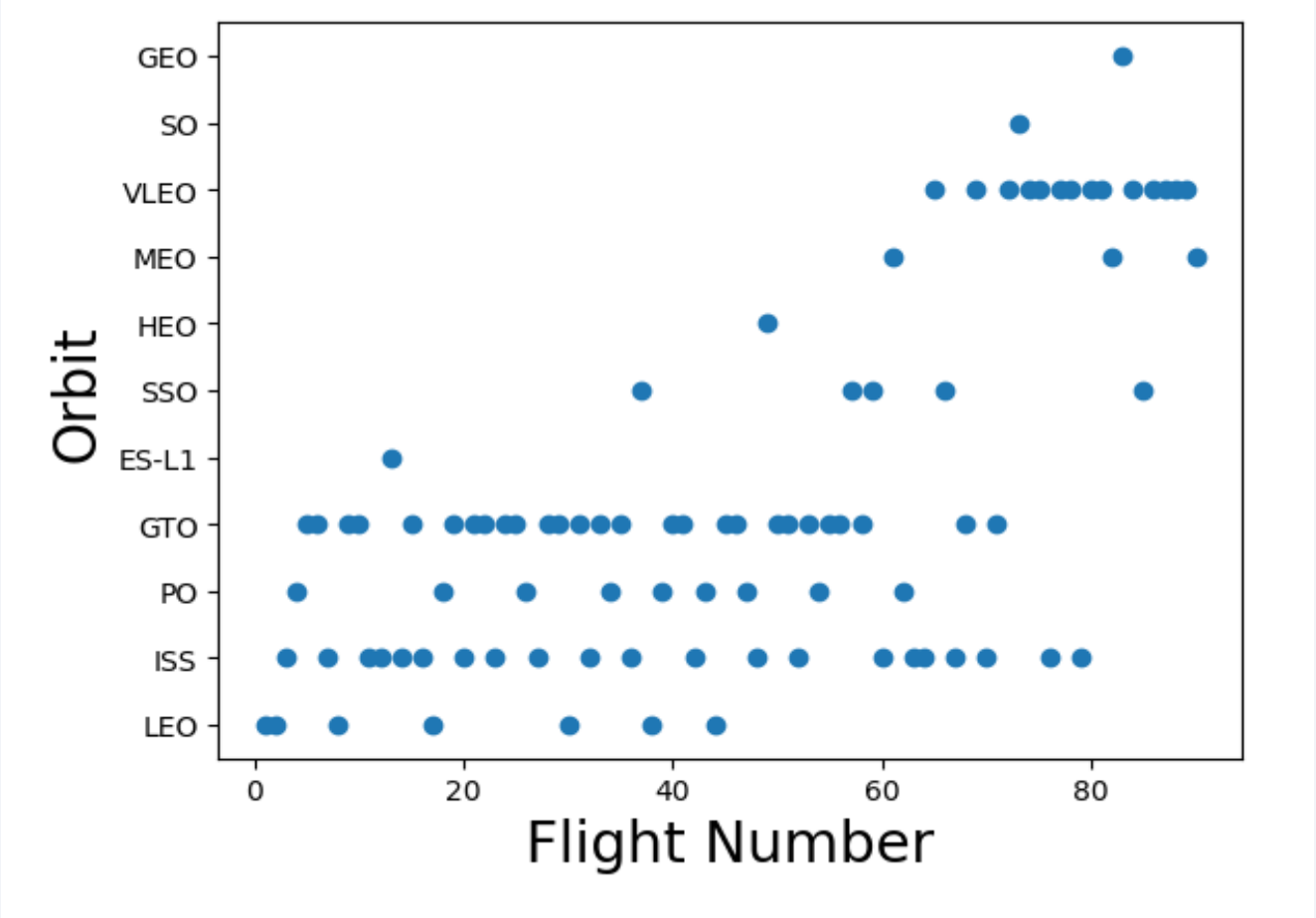
This figure shows the possibility of the orbits to influences the landing outcomes as some orbits have 100% success rate such as SSO, HEO, GEO and ES-L1.

However, some of these orbits have only 1 occurrence such as GEO, SO, HEO and ES-L1 which mean this data need more dataset to see pattern or trend.



Flight Number vs. Orbit Type

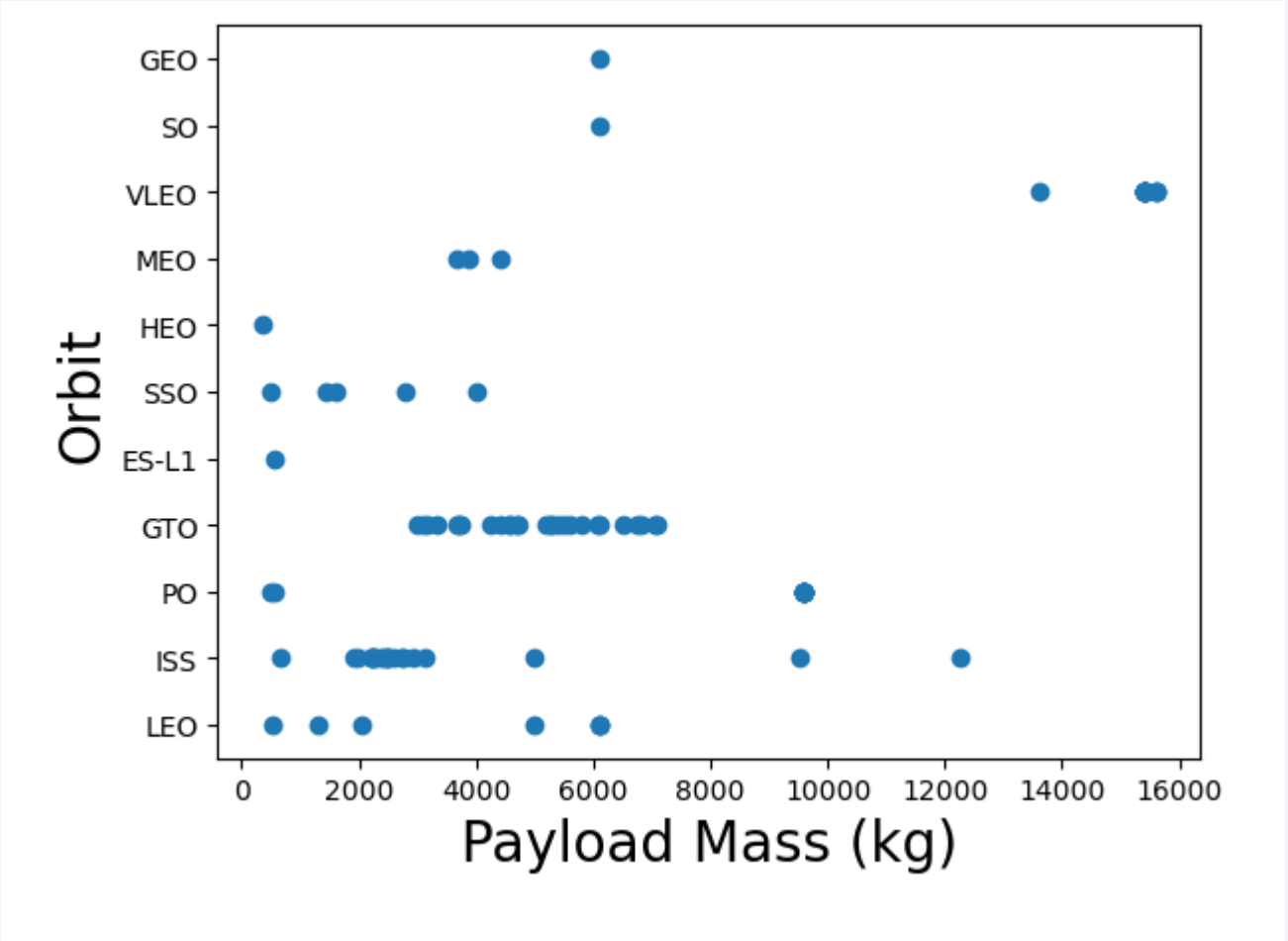
This scatter plot shows that generally the larger the flight number on each orbits, the greater the success rate except for GTO orbit which depicts no relationship between both attributes.



Payload vs. Orbit Type

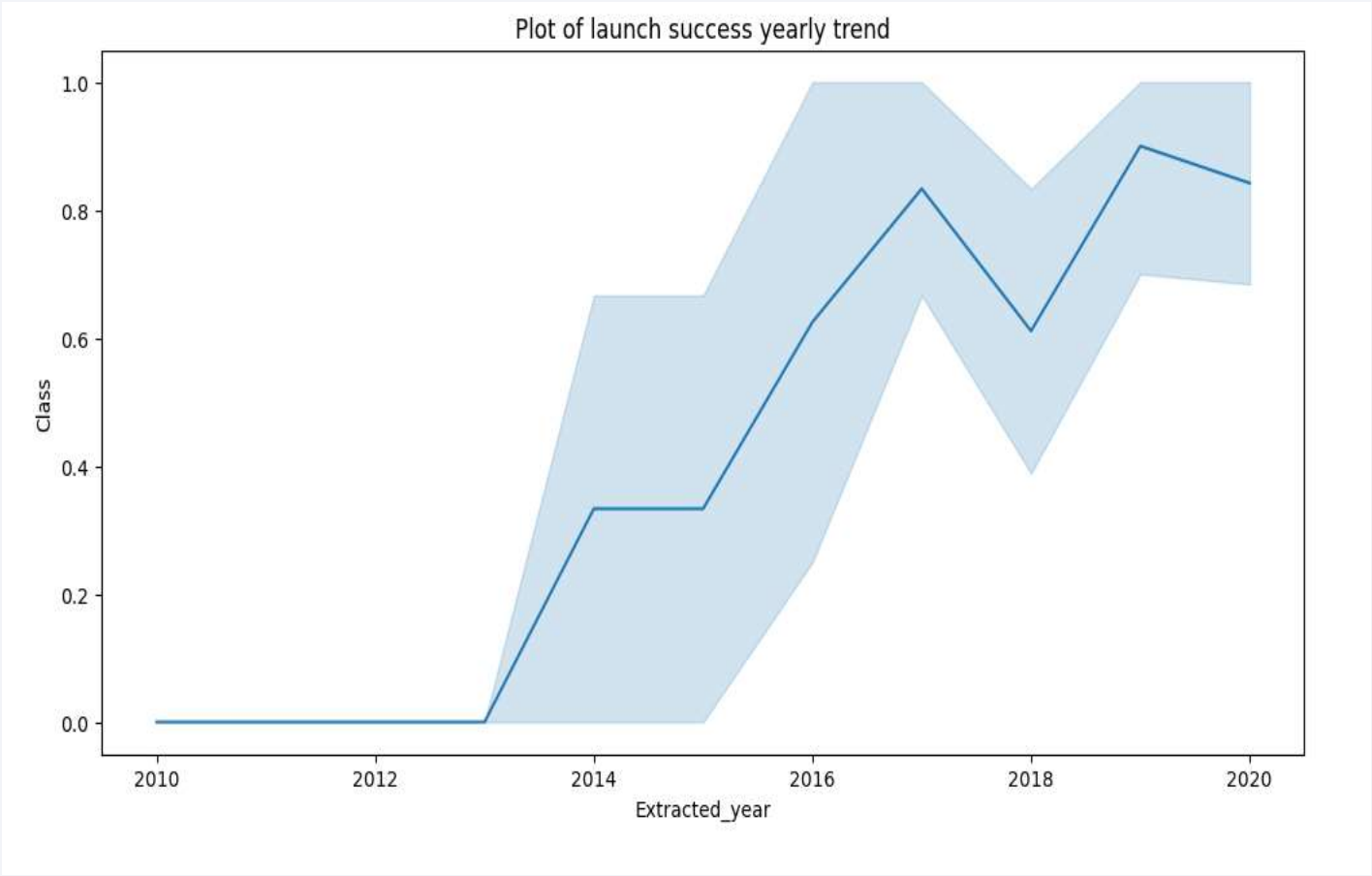
Heavier payload has positive impact on LEO, ISS and PO orbit. However it has negative impact on MEO and VLEO orbit.

GTO orbit seem to depict no relation between the attributes.



Launch Success Yearly Trend

This figure clearly shows and increasing trend from the year 2013 until 2020.



All Launch Site Names

We used the key word DISTINCT to show only unique launch sites from the SpaceX data.

```
In [22]: %sql SELECT DISTINCT LAUNCH_SITE as "Launch_Sites" FROM SPACEXTBL;
 * sqlite:///my_data1.db
Done.
```

Out[22]:

Launch_Sites
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

We used the query above to display 5 records where launch sites begin with CCA.

```
In [23]: %sql SELECT LAUNCH_SITE FROM SPACEXTBL WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5;
 * sqlite:///my_data1.db
Done.
```

Out[23]:

Launch_Site
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40

Total Payload Mass

We calculated the total payload carried by boosters from NASA but the result was none

Display the total payload mass carried by boosters launched by NASA (CRS)

In [24]:
```
%sql SELECT SUM (PAYLOAD_MASS__kg_) FROM SPACEXTBL WHERE CUSTOMER = 'NASA(CRS)' ;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[24]:

SUM (PAYLOAD_MASS__kg_)
None

Average Payload Mass by F9 v1.1

We calculated the average payload mass carried by booster version F9 v1.1 as 2928.4

```
In [36]: %sql SELECT AVG (PAYLOAD_MASS__KG_) FROM SPACEXTBL WHERE BOOSTER_VERSION = 'F9 v1.1';
 * sqlite:///my_data1.db
Done.
Out[36]: AVG (PAYLOAD_MASS__KG_)
         2928.4
```

First Successful Ground Landing Date

We use the min() function to find the result.

We observed that the dates of the first successful landing outcome on ground pad was 22nd December 2015.

```
In [26]: %sql SELECT MIN (DATE) AS "First Successful Landing" FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (ground pad)';
 * sqlite:///my_data1.db
Done.
Out[26]: First Successful Landing
         2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

We used the WHERE to filter for boosters which have successfully landed on drone ship and applied the AND condition to determine successful landing with payload mass greater than 4000 but less than 6000

```
In [27]: %sql SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__
 * sqlite:///my_data1.db
Done.
```

Out[27]:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

We used wildcard like % to filter for WHERE MisssionOutcome was as success or a failure.

```
In [37]: %sql SELECT COUNT (MISSION_OUTCOME) AS "succesful mission "FROM SPACEXTBL WHERE MISSION_OUTCOME LIKE 'Success%'
 * sqlite:///my_data1.db
Done.
```

Out[37]:

succesful mission
100

```
In [38]: %sql SELECT COUNT (MISSION_OUTCOME) AS "failure mission " FROM SPACEXTBL WHERE MISSION_OUTCOME LIKE 'Fail%'
 * sqlite:///my_data1.db
Done.
```

Out[38]:

failure mission
1

Boosters Carried Maximum Payload

```
In [43]: %sql SELECT DISTINCT BOOSTER_VERSION AS "Booster Versions which carried the Maximum Payload Mass" FROM SPACEXTBL \
         WHERE PAYLOAD_MASS__KG_ =(SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);

 * sqlite:///my_data1.db
Done.
```

Out[43]:

Booster Versions which carried the Maximum Payload Mass
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

We determined the booster that have carried the maximum payload using a subquery in the WHERE and the MAX() function.

2015 Launch Records

We used the WHERE, AND conditions to filter for failed landing outcomes in drone ship, their booster versions and launch sites names for year 2015

```
In [56]: %sql select Booster_Version, Launch_Site from SPACEXTBL where Landing_Outcome = 'Failure (drone ship)' and substr(Date,7,4)='2015'
 * sqlite:///my_data1.db
Done.
Out[56]: Booster_Version  Launch_Site
```

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

In [57]:
```
%sql select count(landing_outcome), landing_outcome from SPACEXTBL \
where DATE between '2010-06-04' and '2017-03-20' group by landing_outcome \
order by count(landing_outcome) desc
```

```
 * sqlite:///my_data1.db
Done.
```

Out[57]:

count(landing_outcome)	Landing_Outcome
10	No attempt
5	Success (drone ship)
5	Failure (drone ship)
3	Success (ground pad)
3	Controlled (ocean)
2	Uncontrolled (ocean)
2	Failure (parachute)
1	Precluded (drone ship)

We selected Landing outcomes and the COUNT of landing outcomes from the data and used the WHERE to filter BETWEEN 2010-06-04 to 2017-03-20.

We applied the GROUP BY to group the landing outcomes and the ORDER BY to order the grouped landing outcome in descending order.


Section 3
Launch Sites
Proximities Analysis

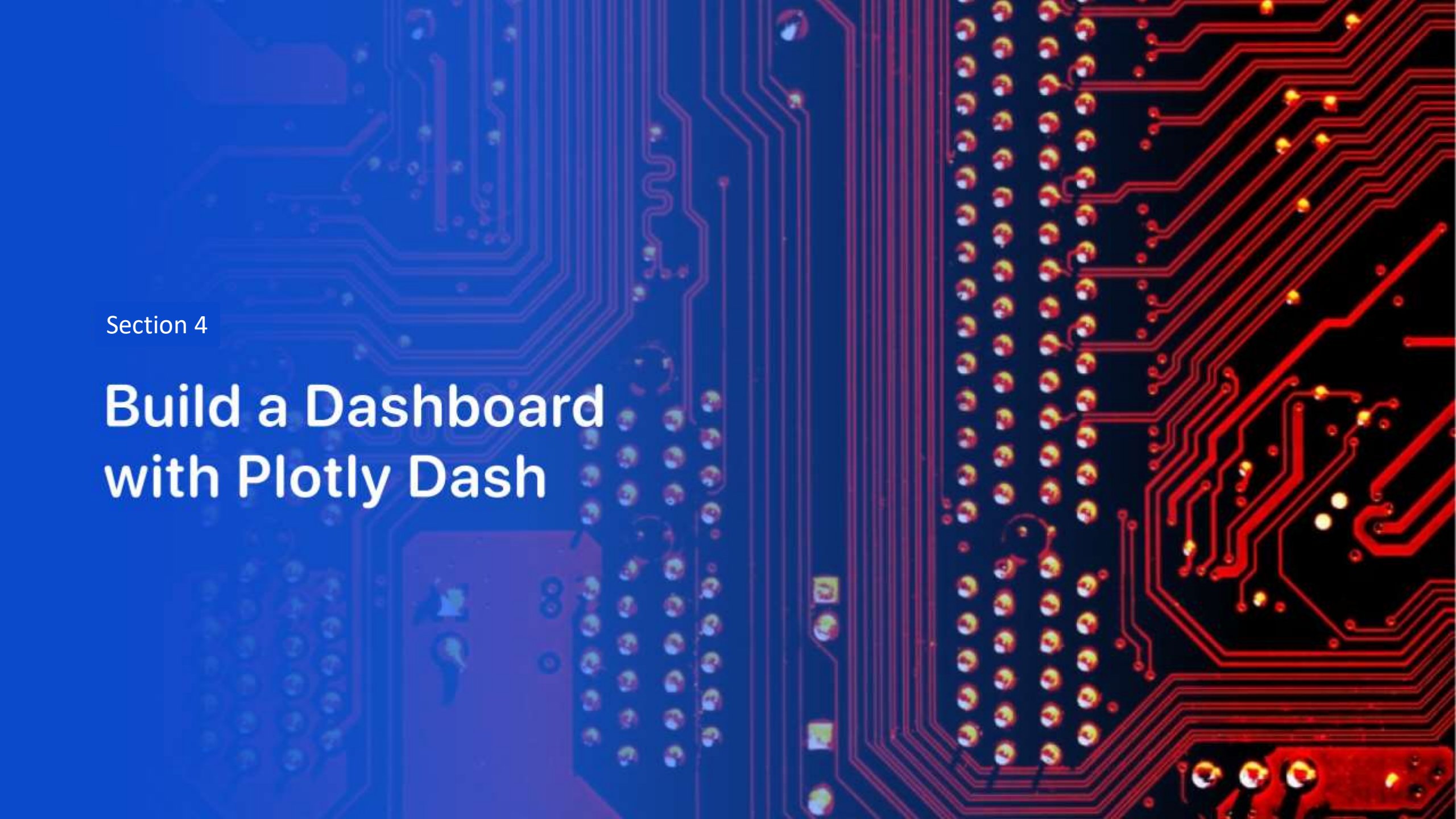
Location of all the Launch Sites

We can see that all the SpaceX launch sites are located inside the United States.

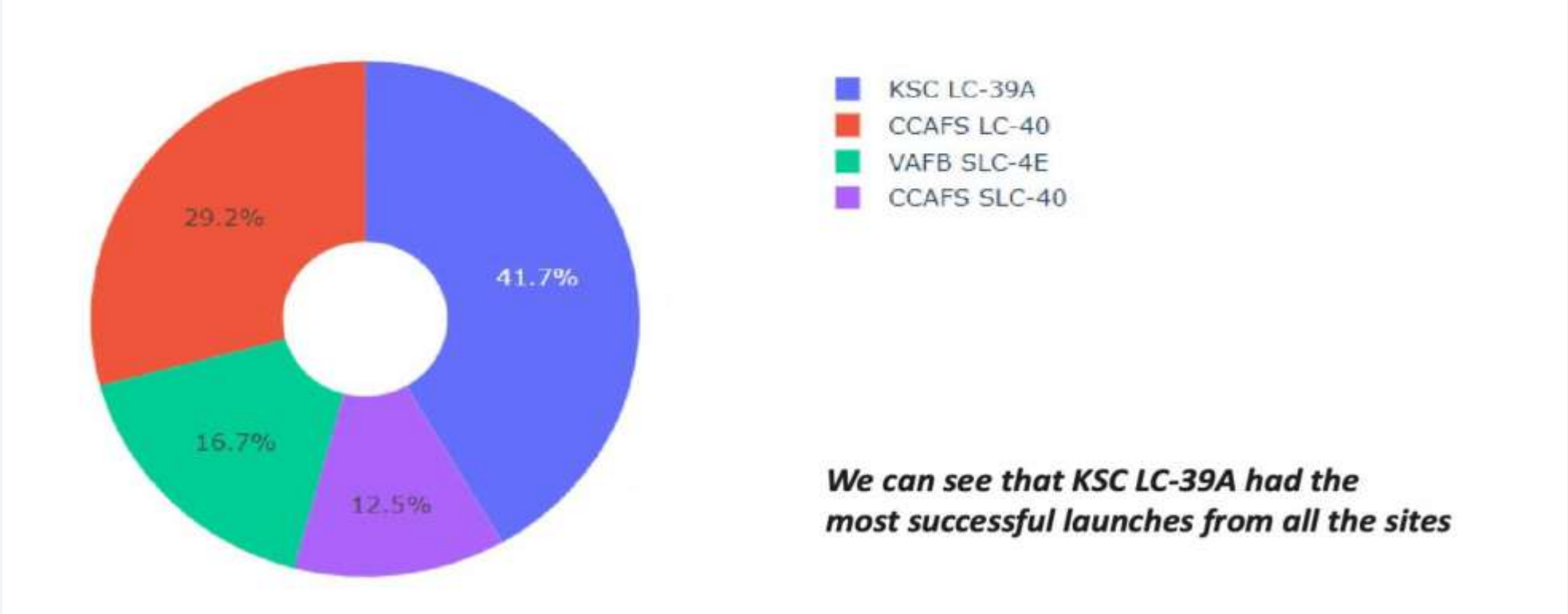
Markers showing launch sites with color labels

Launch Sites Distance to Landmarks

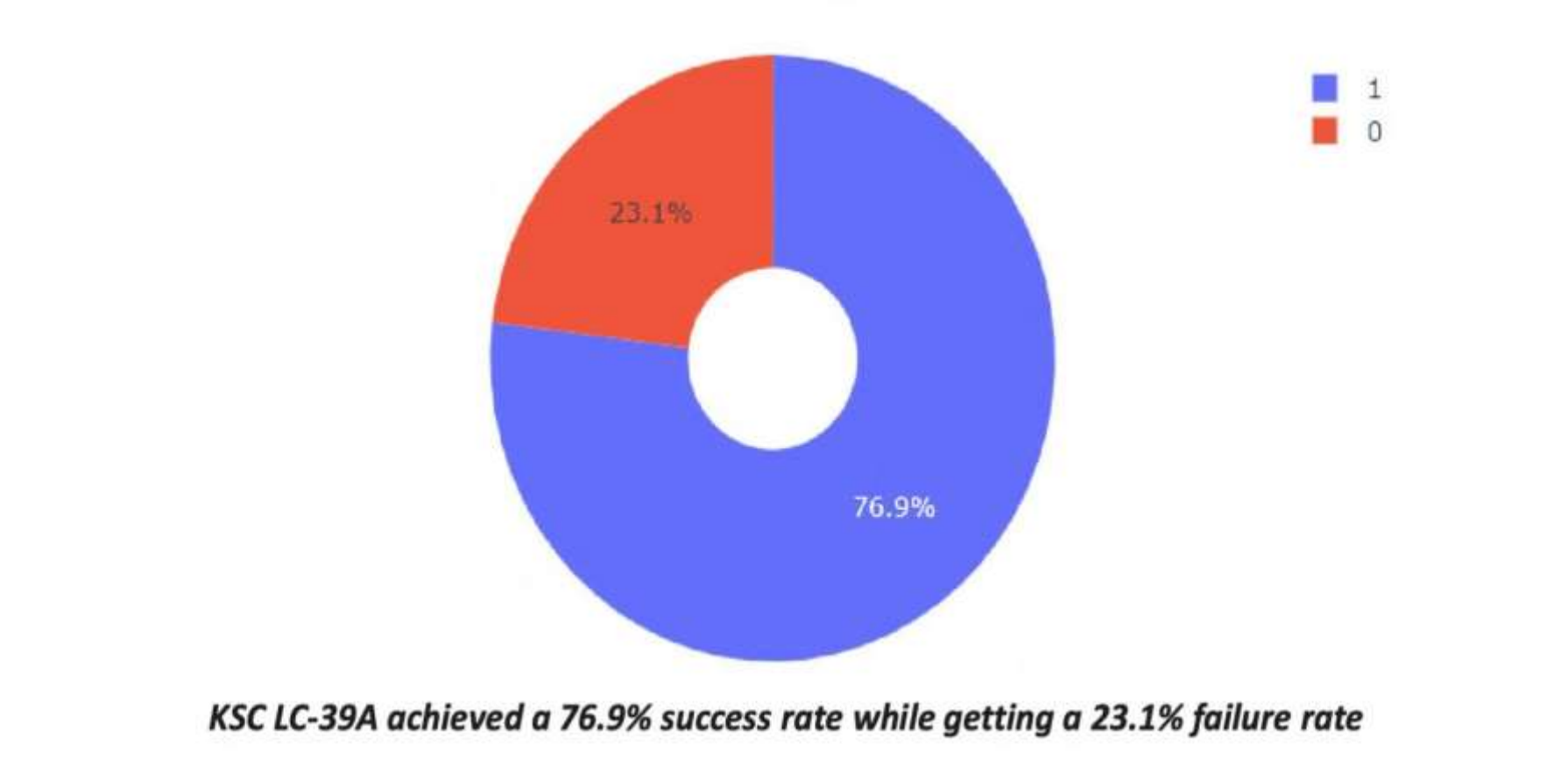
- Are launch sites in close proximity to railways? No
- Are launch sites in close proximity to highways? No
- Are launch sites in close proximity to coastline? Yes
- Do launch sites keep certain distance away from cities? Yes


Section 4
Build a Dashboard with Plotly Dash

The success percentage by each sites

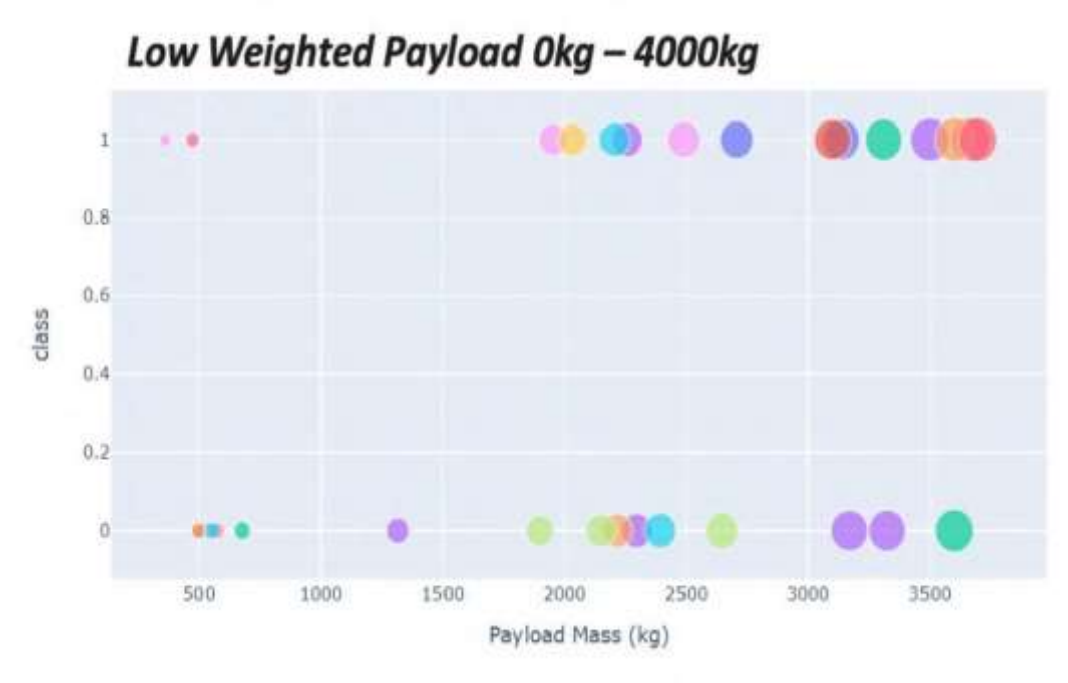


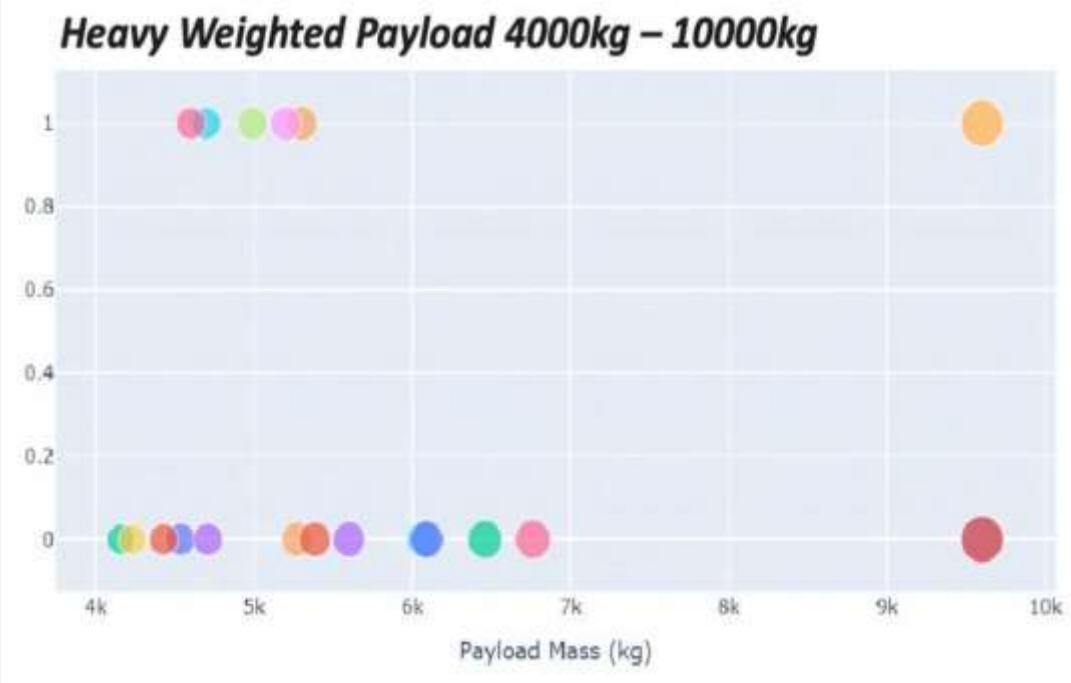
The highest launch-success ratio: KSC LC-39A



KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate

Payload vs Launch Outcome Scatter Plot





Section 5

Predictive Analysis (Classification)

Classification Accuracy

We can see that the best algorithm is the Tree Algorithm which have the highest classification accuracy.

In [32]:

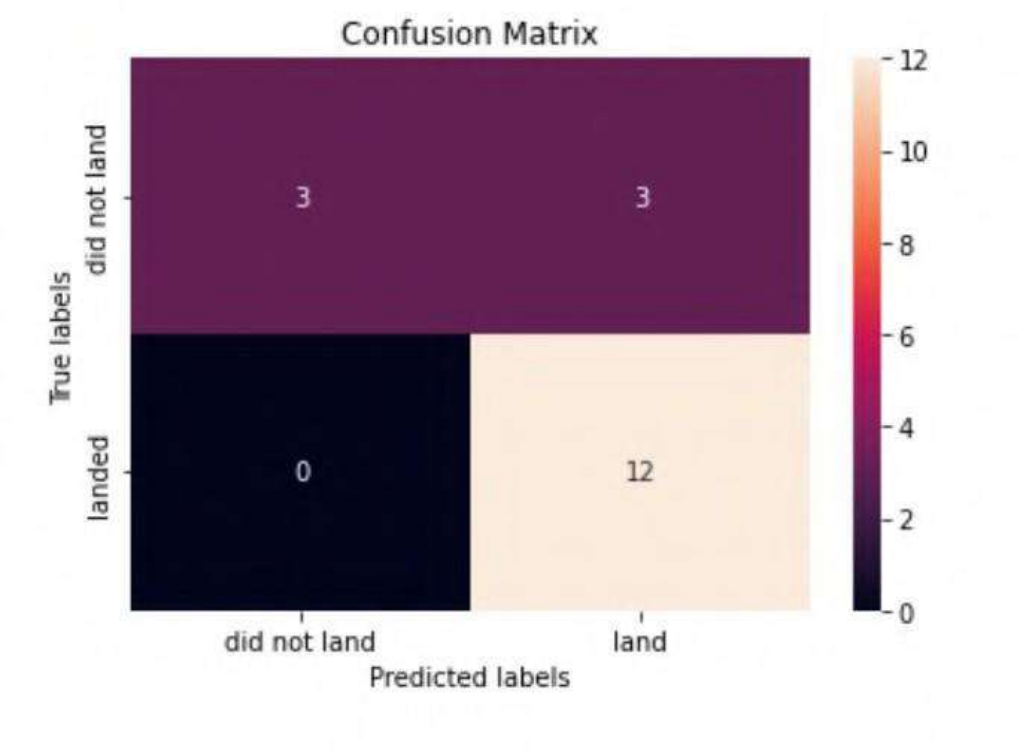
```
models = {'KNeighbors':knn_cv.best_score_,
              'DecisionTree':tree_cv.best_score_,
              'LogisticRegression':logreg_cv.best_score_,
              'SupportVector': svm_cv.best_score_}

bestalgorithm = max(models, key=models.get)
print('Best model is', bestalgorithm,'with a score of', models[bestalgorithm])
if bestalgorithm == 'DecisionTree':
    print('Best params is :', tree_cv.best_params_)
if bestalgorithm == 'KNeighbors':
    print('Best params is :', knn_cv.best_params_)
if bestalgorithm == 'LogisticRegression':
    print('Best params is :', logreg_cv.best_params_)
if bestalgorithm == 'SupportVector':
    print('Best params is :', svm_cv.best_params_)
```

```
Best model is DecisionTree with a score of 0.8607142857142858
Best params is : {'criterion': 'entropy', 'max_depth': 6, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'splitte
r': 'best'}
```

Confusion Matrix

The confusion matrix shows that the classifier can distinguish between the different classes. The major problem is the false positives.



Conclusions

- The Tree Classifier Algorithm is the best Machine Learning approach for this dataset
- The low weighted payloads performed better than heavy weighted payloads
- Starting from the year 2013, the success rate for SpaceX launches is increased
- KSCLC-39A have the most successful launches of any sites
- SSO orbit have the most success rate 100%

Thank you!